

# *Wektorowe bazy danych*

## *Vector databases*

dr hab. inż. Maciej Grzenda, prof. uczelni

[Maciej.Grzenda@pw.edu.pl](mailto:Maciej.Grzenda@pw.edu.pl)

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska  
Wydział Matematyki i Nauk Informacyjnych

# Text and binary content in databases: traditional setting

- In relational databases, text data are stored:
  - If possible, as small text strings (e.g. up to 32767 bytes in Oracle for `nvarchar2` data type)
  - Otherwise, as large text columns such as columns defined with the `text` type in Ms SQL Server and `CLOB` type in Oracle Database (e.g. maximum size of CLOB is  $(4 \text{ gigabytes} - 1) * (\text{database block size})$ )
- Similarly, binary values (such as images) can be stored as:
  - For small values as `binary` (Ms SQL Server) or `RAW` (Oracle Database), with e.g. 32767 bytes for `RAW` in Oracle Database
  - Otherwise as `image` or `BLOB` with .... maximum size of  $(4 \text{ gigabytes} - 1) * (\text{database block size})$ .
- The columns of all these data types (except for small text columns) have limited querying options and/or dedicated APIs (stored procedures). Their primary use was to read and write raw text and binary content, rather than query it

See e.g. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/Data-Types.html> for details on data types supported by Oracle Database 26.

# The challenge of multimedia data querying

- Developments in machine learning methods, including Large Language Models (LLM) and multimodal models, providing a basis for e.g. ChatGPT, raised interest in non-tabular data management, such as text, audio, images, and video management.
- Such data are typically represented by high-dimensional vectors, with possibly billions of them stored and queried.
- But how to store such data?
- How to efficiently search for similar text documents, images, ...?
  - Are RDBMSs with their SQL queries suitable for it? What about Apache Hadoop? Columnar NoSQL platforms?
  - In fact, none of these platforms supports an efficient search for similar text or image content. Even defining what similar means in mathematical terms sounds like a challenge.
  - Hence, vector database management systems were conceived

In practice, applications of vector databases may involve billions of vectors and demand millisecond query latencies [Pan et al., 2024].

# Vector databases and vector database management systems

- Vector databases are used to store ... high-dimensional feature vectors
- Such vectors are used by retrieval search applications, including LLMs, recommendation systems, document retrieval, e-commerce [Pan et al., 2024]
- Vector database management systems are expected to:
  - efficiently answer queries for similar vectors, e.g. similar images.  
However:
    - Similarity of vectors is a vague concept, i.e. what is similar is application-dependent;
    - Similarity computation is more expensive than filtering based on the values of a few attributes (as in SQL).
  - efficiently retrieve possibly large vectors. However, vectors are typically larger than records in relational databases.
  - address the challenge that vectors can not be easily indexed.  
However, vector collections are not sortable.

# Vector databases in practice

- One of two approaches can be followed:
  - Deploying a dedicated vector database, such as:
    - Pinecone (ranked #48 in the db-engines ranking), <https://www.pinecone.io>
    - Milvus (ranked #56 in the db-engines ranking), <https://milvus.io>
    - Qdrant (ranked #65 in the db-engines ranking), <https://qdrant.tech/>
  - Using one of existing database management systems with vector extensions
    - an example is PostgreSQL, ranked #4 according to popularity in the db-engines ranking at <https://db-engines.com/en/ranking> and classified as relational database management system for which vector extensions were developed
- In both cases, the challenge is not only to store vector data, but also to:
  - generate vector representation of e.g. text and images
  - identify similar vectors, i.e. query vectors in terms of their similarity
  - support Retrieval-Augmented Generation (RAG), i.e. systems which answer questions (frequently in the form of prompts) based on the content of your database

Pgvector available at <https://github.com/pgvector/pgvector> is the open source similarity search extension for PostgreSQL making the search operations possible. It supports exact and approximate nearest neighbour search and several distance functions including L2 distance (Euclidean), cosine distance, L1 distance (Manhattan), Hamming distance. See project website for details.

# Vector databases – first steps

- Let us create a sample table with vectors in PostgreSQL with the pgvector extension. See appendix of this presentation for details on how to set up the PostgreSQL instance

```
mydb=# CREATE TABLE first_items (id serial PRIMARY KEY, embedding vector(3), name
varchar(20));
INSERT INTO first_items (embedding,name) VALUES ('[1,2,3]','1st'),
('[4,5,6]','2nd'),('[2,2,3]','3rd'),
('[7,8,9]','last');
```

- Now let us execute the queries:

```
mydb=# SELECT * FROM first_items ORDER BY embedding <-> '[3,2,2.5]' LIMIT 2;
 id | embedding | name
----+-----+-----
  3 | [2,2,3]   | 3rd
  1 | [1,2,3]   | 1st
(2 rows)
```

```
mydb=# SELECT * FROM first_items ORDER BY embedding <=> '[3,2,2.5]' LIMIT 2;
 id | embedding | name
----+-----+-----
  4 | [7,8,9]   | last
  3 | [2,2,3]   | 3rd
```

<-> denotes L2 distance (Euclidean distance). Cosine distance is obtained by <=>.

# Vector databases – converting text to vectors

- Let us convert our text data into vectors. We will use JupyterLab notebook and Python language for simplicity.
  - First let us install sample library of sentence-transformers (**SBERT**) documented at <https://sbert.net> . It can be used to compute **embeddings** (pol. **osadzenia, zanurzenia**) from text, images, audio, or video

```
!pip install sentence-transformers
```

- Now let us use it to calculate embeddings:

```
from sentence_transformers import SentenceTransformer
# Load the model (downloads automatically on first run)
model = SentenceTransformer("all-MiniLM-L6-v2")

# Example texts
sentences = [
    "Let us create a sample table with vectors in PostgreSQL.",
    "Triggers are procedures executed by DBMS.",
    "Capabilities may depend on SQL implementation."
]
# Generate embeddings
embeddings = model.encode(sentences)

# Print results
for i, emb in enumerate(embeddings):
    print(f"Sentence {i}:")
    print(f"Text: {sentences[i]}")
    print(f"Embedding length: {len(emb)}")
    print(f"First 5 values: {emb[:5]}\n")
```

# Vector databases – converting text to vectors

```
from sentence_transformers import
SentenceTransformer

# Load the model (downloads automatically on
first run)
model = SentenceTransformer("all-MiniLM-L6-
v2")

# Example texts
sentences = [
    "Let us create a sample table with
vectors in PostgreSQL.",
    "Triggers are procedures executed by
DBMS.",
    "Capabilities may depend on SQL
implementation."
]

# Generate embeddings
embeddings = model.encode(sentences)

# Print results
for i, emb in enumerate(embeddings):
    print(f"Sentence {i}:")
    print(f"Text: {sentences[i]}")
    print(f"Embedding length: {len(emb)}")
    print(f"First 5 values: {emb[:5]}\n")
```

Sentence 0:

Text: Let us create a sample table with  
vectors in PostgreSQL.

**Embedding length: 384**

**First 5 values: [ 0.01161648 -0.06005547 -0.06279646 -  
0.03293108 -0.04404429]**

Sentence 1:

Text: Triggers are procedures executed by  
DBMS.

**Embedding length: 384**

**First 5 values: [ 0.03163411 -0.04178087 -0.03714243  
0.01466281 -0.00180273]**

Sentence 2:

Text: Capabilities may depend on SQL  
implementation.

**Embedding length: 384**

**First 5 values: [ 0.0277023 -0.04269028 -0.08593024 -  
0.00449728 -0.0852515 ]**

These are the first 5 elements of 384-  
dimensional embedding of the sentence. The  
embedding was calculated by the SBERT model  
all-MiniLM-L6-v2. In this case, embedding can  
be treated as a function  $f$ : sentence  $\rightarrow R^{384}$



# Vector databases – querying text content

- Let us create a more realistic table for vector data

```
CREATE TABLE documents (id serial  
PRIMARY KEY, embedding vector(384),  
raw_text varchar(100));
```

- Now let us use it to insert into the table the records with embeddings of these sentences:

- "Let us create a sample table with vectors in PostgreSQL.",
- "Triggers are procedures executed by DBMS.",
- "Capabilities may depend on SQL implementation.",
- "Please write this text on the table.",
- "The poem written on the table is by W. Shakespeare"

- We will look for records most similar to these sentences (note typos)

```
new_sentences = [  
    "The poetry really matters.",  
    "Multiple records can be inserted into  
one ttable of ralational database"  
]
```

- First, we need to calculate the embedding for each sentence from new\_sentences
- Now we can use e.g. cosine similarity to find the most similar records by comparing:
  - the embedding of one of new sentences
  - to the embeddings present in the documents table

# Vector databases – querying text content

- Assuming we have records with embeddings of these sentences:

- "Let us create a sample table with vectors in PostgreSQL.",
- "Triggers are procedures executed by DBMS.",
- "Capabilities may depend on SQL implementation.",
- "Please write this text on the table.",
- "The poem written on the table is by W. Shakespeare"

- And we look for records most similar to these sentences (note typos)

```
new_sentences = [  
    "The poetry really matters.",  
    "Multiple records can be inserted into  
one ttable of ralational database"  
]
```

```
for sentence in sentences:  
    embedding = model.encode(sentence).tolist()  
    cursor.execute("""  
        SELECT raw_text, 1 - (embedding <=>  
        %s::vector) AS cosine_similarity  
        FROM documents  
        ORDER BY embedding <=> %s::vector  
        LIMIT 3;  
        """, (embedding, embedding))  
  
    print(f"\nThe vectors most similar to:  
'{sentence}' are:")  
    for raw_text, similarity in cursor.fetchall():  
        print(f" {similarity:.4f} | {raw_text}")
```



The vectors most similar to: 'The poetry really matters.' are:  
0.3807 | The poem written on the table is by W. Shakespeare  
0.1349 | Please write this text on the table.  
0.0361 | Capabilities may depend on SQL implementation.

The vectors most similar to: 'Multiple records can be inserted into one ttable of ralational database' are:  
0.2626 | Capabilities may depend on SQL implementation.  
0.2296 | Triggers are procedures executed by DBMS.  
0.2046 | Let us create a sample table with vectors in PostgreSQL.

# The challenges related to querying vector data

- Queries aimed at finding records with most similar vectors can rely on:
  - Exact search, also known as k-nearest neighbor (kNN). This can be expensive
  - or approximate vector search, i.e. approximate nearest neighbour
- Vector indexes relying on graphs can be used to reduce computational cost
- Different indexing strategies have been proposed. Further details can be found here:
  - Ms SQL Server: <https://learn.microsoft.com/en-us/sql/sql-server/ai/vectors?view=sql-server-ver17>
  - PostgreSQL/pgvector: <https://github.com/pgvector/pgvector>
  - Oracle Database: <https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/guidelines-using-vector-indexes.html>
- Indexing approaches include:
  - The use of graphs
  - The use of lists to divide vectors into lists and search only subsets of lists closest to the vector of interest
- Note that a separate index is typically created for each distance function that we plan to use

## Further reading

- [Pan2024] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Vector Database Management Techniques and Systems. In Companion of the 2024 International Conference on Management of Data (SIGMOD '24). Association for Computing Machinery, New York, NY, USA, 597–604. <https://doi.org/10.1145/3626246.3654691>
- <https://www.oracle.com/database/vector-database/>
- <https://learn.microsoft.com/en-us/sql/sql-server/ai/vectors?view=sql-server-ver17>

# Summary

- Vector databases have revolutionised the way text, images, audio, and video can be queried
- They enable semantic search rather than exact match-based or querying based on patterns available with LIKE
- New challenges related to running vector databases include:
  - Calculating embeddings
  - Efficient similarity search using different distance functions
  - Development of Retrieval-Augmented Generation (RAG) solutions

# Appendix I

## Setting up Docker container with PostgreSQL instance

# Setting up PostgreSQL environment – part I

- If you want to run the same setting, follow these steps:

- install Docker, if needed
- run `docker build -t postgres-pgvector .`
- start docker container using:

```
docker run \  
  --name postgres-vector-experiments \  
  -e POSTGRES_PASSWORD=mySecretPassword \  
  -d \  
  -p 5432:5432 \  
  postgres-pgvector
```

- connect to the PostgreSQL instance, configure your first database in select it as the current one (commands to execute shown in bold):

```
(base) user@myComputer % docker exec -it postgres-vector-experiments psql -U postgres  
psql (16.14 (Debian 16.14-1.pgdg12+1))  
Type "help" for help.  
  
postgres=# CREATE DATABASE mydb;  
\c mydb  
CREATE DATABASE  
You are now connected to database "mydb" as user "postgres".  
mydb=#
```

# Setting up PostgreSQL environment – part II

- The next step, also to be executed within your session with PostgreSQL is to enable the pg-vector extension (already installed as defined in the container definition):

```
mydb=# CREATE EXTENSION vector;  
CREATE EXTENSION  
mydb=#
```

- connect to the PostgreSQL instance, configure your first database in select it as the current one (commands to execute shown in bold):

```
(base) user@myComputer % docker exec -it postgres-vector-experiments psql -U postgres  
psql (16.14 (Debian 16.14-1.pgdg12+1))  
Type "help" for help.  
  
postgres=# CREATE DATABASE mydb;  
\c mydb  
CREATE DATABASE  
You are now connected to database "mydb" as user "postgres".  
mydb=#
```